Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Seguridad de la Información • CE-1115

Taller de Criptografía Aplicada

Chat cifrado entre dos procesos locales con criptografía híbrida

SecureChat: descripción del protocolo

¿Qué viaja por el socket y qué nunca sale de cada extremo?

RSA-OAEP • AES-256-GCM • Handshake • Nonce • Authentication tag

Repositorio: `https://github.com/Adriel23456/SecureChat_Workshop`

PROFESOR

MSc. Andrés F. Vargas Rivera

INTEGRANTES — TRABAJO EN PAREJAS

Nombre	Carné
Adriel S. Chaves Salazar	2021031465
Henry D. Nuñez Pérez	2022089224

II Semestre, 2026 • [Fecha de entrega] • Cartago, Costa Rica

Índice

1 Introducción y objetivo **1**
- 1.1 Alcance . . . 1
- 1.2 Entorno de desarrollo . . . 1

2 Arquitectura general **1**
- 2.1 Organización del código . . . 1
- 2.2 Concurrencia . . . 2

3 Material criptográfico **2**
- 3.1 Par de llaves asimétricas . . . 2
- 3.2 Llave pública y llave privada: qué hace cada una . . . 2
- 3.3 Huella de la llave pública . . . 3
- 3.4 Distribución de las llaves públicas . . . 3
- 3.5 Uso del par de llaves de cada participante . . . 3
- 3.6 Clave de sesión . . . 3
- 3.7 Qué es público, qué es secreto y qué viaja por la red . . . 4

4 Establecimiento de la sesión (handshake) **4**
- 4.1 Mensajes del handshake . . . 5
- 4.2 Generación y protección de la clave de sesión . . . 5
- 4.3 Transcript, firma y confirmación . . . 6
- 4.4 Autenticación mutua . . . 6
- 4.5 Manejo de errores . . . 6

5 Protección de los mensajes **7**
- 5.1 AES-256-GCM . . . 7
- 5.2 Generación del nonce . . . 7
- 5.3 Verificación en el receptor . . . 8
- 5.4 Protección contra repetición . . . 8
- 5.5 Evidencia en pantalla . . . 8
- 5.6 Pruebas automatizadas . . . 8

6 Formato de los mensajes transmitidos **8**

7 Conversación bidireccional y cierre de sesión **9**
- 7.1 Flujo de un mensaje . . . 9
- 7.2 Presentación en consola . . . 9
- 7.3 Cierre controlado . . . 9

8 Detección de manipulación **10**

9 Análisis de seguridad **10**
- 9.1 Modelo de adversario . . . 10
- 9.2 Cumplimiento de los requisitos mínimos . . . 10
- 9.3 Limitaciones conocidas . . . 11

10 Preguntas de cierre **11**

11 Evidencias de la demostración **11**

Referencias **13**

A Ejecución y estructura del repositorio **14**

B Declaración de uso de IA **14**

1. Introducción y objetivo

SecureChat es una aplicación de consola que mantiene una conversación bidireccional entre dos procesos que se ejecutan en la misma computadora, conectados por un socket TCP sobre `localhost`. La criptografía **asimétrica** se usa para establecer la sesión y la criptografía **simétrica autenticada** protege cada mensaje.

[**PENDIENTE:** *Resumen de la solución final en un párrafo: lenguaje, biblioteca, algoritmos, tamaños de llave y resultado de la demostración.*]

1.1. Alcance

[**PENDIENTE:** *Qué cubre y qué no cubre la solución (por ejemplo, autenticación de la llave pública, persistencia, más de dos participantes).*]

1.2. Entorno de desarrollo

Tabla 1.1: Entorno de desarrollo y demostración

Elemento	Detalle
Sistema operativo	Windows [PEND.]
Lenguaje	Python [PEND.]
Biblioteca criptográfica	`cryptography` [PEND.]
Comunicación	Sockets TCP/IP sobre `127.0.0.1`, puerto [PEND.]
Repositorio	`https://github.com/Adriel23456/SecureChat_Workshop`

2. Arquitectura general

El mismo programa se ejecuta dos veces, una instancia por participante (figura 2.1). Bob espera conexiones y Alice inicia la conexión.

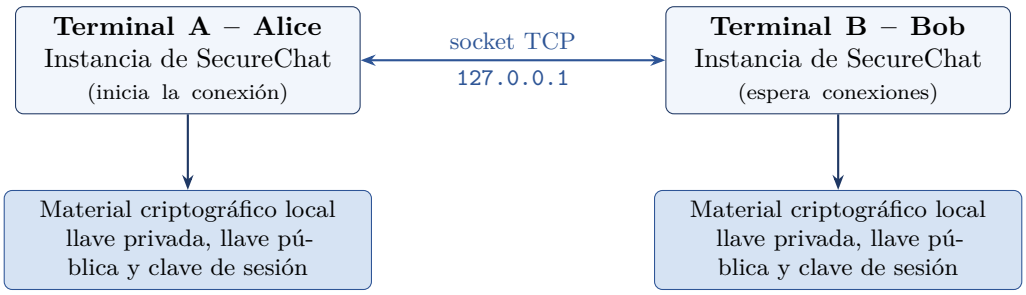


Figura 2.1: Arquitectura de SecureChat: dos instancias del mismo programa conectadas por TCP en la misma computadora.

2.1. Organización del código

[**PENDIENTE:** *Módulos finales y responsabilidad de cada uno (red, framing, llaves, handshake, cifrado, interfaz de consola).*]

2.2. Concurrencia

[**PENDIENTE:** *Cómo se reciben mensajes mientras el usuario escribe (hilo receptor y entrada de teclado) y por qué en Windows no se usa* `select()` *sobre* `stdin`.]

3. Material criptográfico

Cada participante dispone de su propio par de llaves RSA, generado y guardado dentro de su carpeta (`srcAlice/keys/` y `srcBob/keys/`). Ninguna de estas carpetas se versiona: la regla `keys/` del `.gitignore` las excluye del repositorio.

3.1. Par de llaves asimétricas

Tabla 3.1: Parámetros del par de llaves asimétricas

Parámetro	**Valor**
Algoritmo	RSA
Tamaño del módulo	3072 bits (se rechaza al cargar cualquier llave menor a 2048 bits)
Exponente público	$e = 65537$
Generación	`rsa.generate_private_key` de `cryptography`, con el generador aleatorio seguro del sistema operativo
Relleno para cifrar	OAEP con SHA-256 y MGF1(SHA-256) (uso en el handshake, apartado 4)
Relleno para firmar	PSS con SHA-256 y MGF1(SHA-256)
Formato de la llave privada	PEM, PKCS#8, **cifrada** con una frase de paso (`BestAvailableEncryption`)
Formato de la llave pública	PEM, *SubjectPublicKeyInfo* (X.509)
Frase de paso	Mínimo 8 caracteres; se pide sin eco en pantalla (`getpass`) y nunca se guarda
Archivos	`keys/private_key.pem` y `keys/public_key.pem` dentro de la carpeta de cada participante

Se eligió RSA-3072 en lugar del mínimo de 2048 bits porque ofrece alrededor de 128 bits de seguridad, un nivel que se mantiene vigente a largo plazo. El costo es una generación de llaves algo más lenta (uno o dos segundos), que ocurre una sola vez. El exponente $e = 65537$ es el valor estándar: verificar y cifrar con él es rápido y no presenta las debilidades de los exponentes muy pequeños.

3.2. Llave pública y llave privada: qué hace cada una

Las dos llaves están relacionadas matemáticamente: lo que una hace, solo la otra lo deshace. La llave pública (n, e) se deriva de la privada, pero el camino inverso exige factorizar n, lo cual es inviable para un módulo de 3072 bits. Por eso:

- **La llave pública puede compartirse.** Sirve para *cifrar hacia* su dueño y para *verificar* sus firmas. Conocerla no permite descifrar ni firmar.
- **La llave privada debe permanecer bajo control exclusivo de su dueño.** Es la única que puede *descifrar* lo cifrado con la pública y *firmar*. Nunca se muestra en pantalla, nunca se escribe sin cifrar y nunca viaja por el socket.

- **La llave pública no reemplaza a la privada.** Con la pública de Bob, Alice puede proteger la clave de sesión para que solo Bob la lea, pero no puede leerla ella misma una vez cifrada ni hacerse pasar por Bob. Si la pública bastara para descifrar, cualquiera que la conociera (es decir, todos) podría hacerlo.

Prueba de correspondencia. La opción *Verificar mis llaves* del menú carga ambos archivos y ejecuta tres comprobaciones: (1) el módulo n y el exponente e derivados de la privada coinciden con los de la pública; (2) una firma PSS hecha con la privada es aceptada por la pública; y (3) un valor aleatorio cifrado con OAEP usando la pública se recupera con la privada. El valor de prueba es aleatorio, se descarta y solo se informa "aprobado" o "fallido".

3.3. Huella de la llave pública

La huella es el resumen SHA-256 de la codificación DER de la llave pública: 32 bytes que se muestran como `SHA256:ab:cd:...`. Una llave de 3072 bits ocupa cientos de bytes y no se puede comparar a simple vista; la huella sí. Además, si un atacante sustituye la llave pública durante el handshake, la huella cambia por completo.

La huella es información pública: mostrarla no revela ningún secreto. Es lo único de las llaves que el programa muestra en pantalla.

3.4. Distribución de las llaves públicas

Decisión. Se adoptó un modelo mixto. La llave pública viaja por el socket durante el handshake, pero antes de usarla el receptor compara su huella SHA-256 con una huella de confianza registrada previamente. Para registrarla, cada participante muestra su huella en su terminal y el otro la copia por un canal independiente de la red (*fuera de banda*). Si las huellas no coinciden, el handshake se aborta. Así se cierra la puerta al atacante en el medio que sustituye la llave pública, un riesgo que la criptografía por sí sola no resuelve si no se autentica la llave.

Las huellas de confianza se guardan en `srcBob/keys/trusted_alice_fingerprint.txt` y `srcAlice/keys/trusted_bob_fingerprint.txt`.

3.5. Uso del par de llaves de cada participante

- **Par de Bob:** Alice cifra la clave de sesión con la llave pública de Bob (RSA-OAEP) y solo Bob puede recuperarla con su privada. Que Bob logre confirmar la sesión con esa clave demuestra que posee la privada correspondiente a la huella registrada.
- **Par de Alice:** Alice firma su mensaje del handshake con su llave privada (RSA-PSS) y Bob verifica la firma con la llave pública de Alice, previamente autenticada por su huella. Así Bob sabe que la clave de sesión la generó Alice y no un tercero.

Con este esquema la autenticación es **mutua** y cada par de llaves cumple una función concreta. [**PENDIENTE:** *Confirmar con la implementación del Paso 3.*]

3.6. Clave de sesión

La clave de sesión K es un secreto simétrico de 256 bits generado por Alice con el generador aleatorio criptográfico del sistema operativo (`AESGCM.generate_key`) al inicio de cada conversa-

ción. Nunca está escrita en el código, nunca se imprime y solo viaja cifrada con RSA-OAEP. Vive únicamente en la memoria de ambos procesos durante la sesión y se descarta al cerrarla (apartado 4).

3.7. Qué es público, qué es secreto y qué viaja por la red

Tabla 3.2: Clasificación del material y de los datos del protocolo

Elemento	Dónde vive	¿Viaja por el socket?	Nivel
Llave privada (Alice, Bob)	`keys/private_key.pem`, cifrada; en memoria solo mientras se usa	Nunca	**Secreto**
Frase de paso	Solo en la memoria de su dueño	Nunca	**Secreto**
Llave pública	`keys/public_key.pem`	Sí, en el handshake	**Público**
Huella de la llave pública	Pantalla y archivo de confianza del par	No (se compara fuera de banda)	**Público**
Clave de sesión	Memoria de ambos procesos	Solo cifrada con RSA-OAEP	**Secreto**
Plaintext del usuario	Solo en la terminal local	Nunca (tras el handshake)	**Secreto**
Nonce	Generado por mensaje	Sí, junto al ciphertext	**Público**
Ciphertext	En tránsito	Sí	**Público**
Authentication tag	En tránsito	Sí	**Público**
Metadatos (puertos, tamaños, horas)	Encabezados TCP/IP	Sí	**Público**

4. Establecimiento de la sesión (handshake)

Antes de permitir cualquier mensaje de usuario, ambas instancias completan un handshake de seis mensajes (figura 4.1). La criptografía asimétrica cumple aquí dos funciones: **proteger** la clave de sesión (RSA-OAEP con la llave pública de Bob) y **autenticar** a los participantes (huellas verificadas fuera de banda y firma RSA-PSS de Alice). Al terminar, ambos poseen la misma clave de sesión K de 256 bits sin que esta haya viajado nunca en texto plano.

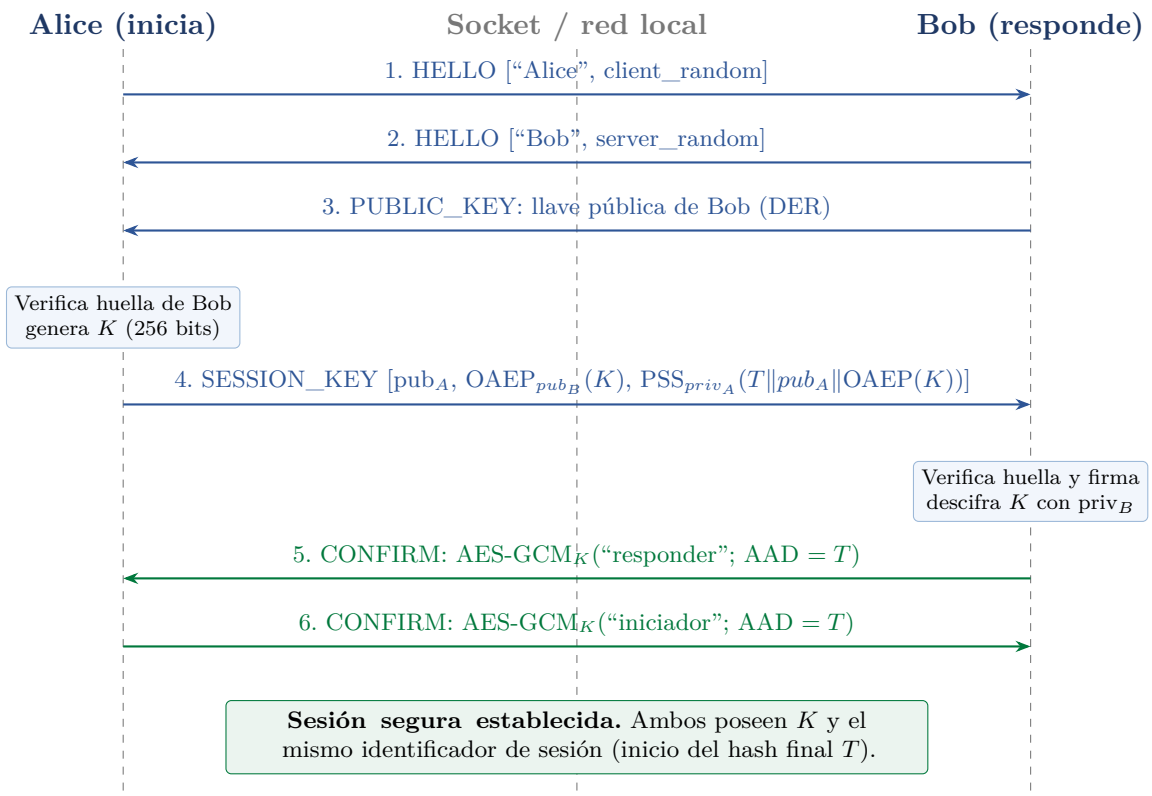


Figura 4.1: Handshake de SecureChat. T es el hash SHA-256 acumulado de todos los mensajes anteriores (transcript).

4.1. Mensajes del handshake

Cada mensaje es una trama con 1 byte de tipo seguido del cuerpo. Los cuerpos con varios valores binarios se codifican como una lista de campos, cada uno precedido por su largo en 4 bytes (big-endian).

Tabla 4.1: Mensajes del handshake

#	Emisor	Tipo (byte)	Contenido y protección
1	Alice	HELLO (1)	Nombre y *client_random* de 32 bytes. Público.
2	Bob	HELLO (1)	Nombre y *server_random* de 32 bytes. Público.
3	Bob	PUBLIC_KEY (4)	Llave pública RSA-3072 de Bob en DER (423 bytes). Pública; Alice la valida con la huella registrada.
4	Alice	SESSION_KEY (5)	Llave pública de Alice (DER); K cifrada con RSA-OAEP (384 bytes); firma RSA-PSS de Alice sobre $T\|pub_A\|\text{OAEP}(K)$ (384 bytes).
5	Bob	CONFIRM (6)	Nonce de 12 bytes y la etiqueta “responder” cifrada con AES-256-GCM usando K y AAD $= T$ (incluye tag de 16 bytes).
6	Alice	CONFIRM (6)	Igual, con la etiqueta “iniciador”.
—	Cualquiera	ABORT (7)	Motivo del fallo en texto. Se envía antes de cerrar la conexión.

4.2. Generación y protección de la clave de sesión

Alice genera K con `AESGCM.generate_key(bit_length=256)`, que usa el generador aleatorio criptográfico del sistema operativo. La clave es nueva en cada sesión y no está escrita en el

código. Luego la cifra con la llave pública de Bob mediante RSA-OAEP (SHA-256, MGF1 con SHA-256); solo la llave privada de Bob puede recuperarla. Antes de enviar el mensaje, el programa comprueba que los 32 bytes de K no aparecen en los bytes que salen por el socket.

4.3. Transcript, firma y confirmación

Ambos lados calculan por separado un hash SHA-256 acumulado (T) de todos los mensajes del handshake, en orden. Este hash cumple tres funciones:

- **Firma de Alice.** Alice firma $T \| pub_A \| \text{OAEP}(K)$. Como T incluye los dos valores aleatorios y la llave pública de Bob, la firma solo es válida para *esta* sesión y no se puede reutilizar en otra.
- **Confirmación de clave.** Cada lado cifra una etiqueta fija con AES-GCM usando K y T como datos asociados (AAD). Si las claves o los transcripts difieren, la verificación del tag falla. Bob confirma primero (demuestra que descifró K) y Alice después.
- **Identificador de sesión.** Los primeros 8 bytes del hash final se muestran en ambas terminales; deben coincidir.

4.4. Autenticación mutua

Bob se autentica ante Alice porque su llave pública coincide con la huella registrada fuera de banda y porque su confirmación demuestra que posee la llave privada correspondiente. Alice se autentica ante Bob por su huella y por su firma RSA-PSS. Ninguna llave privada viaja por el socket.

4.5. Manejo de errores

Tabla 4.2: Fallos del handshake y reacción

Fallo	Reacción
Llave pública mal formada, no RSA o menor a 2048 bits	ABORT y cierre.
Huella recibida distinta de la registrada	ABORT (posible intermediario); se muestran ambas huellas.
Firma de Alice inválida	ABORT y cierre.
Descifrado RSA-OAEP fallido (`ValueError`)	ABORT y cierre.
Confirmación con tag inválido (`InvalidTag`)	ABORT y cierre.
Mensaje fuera de orden (por ejemplo, CHAT antes de HELLO)	ABORT y cierre.
Tiempo de espera agotado (15 s por mensaje)	ABORT y cierre.
Frase de paso incorrecta o falta la huella del par	No se abre la conexión.

En ningún caso se continúa sin una sesión establecida: el chat solo se habilita después de verificar ambas confirmaciones.

Limitación y mitigación. Si la llave pública de Bob se aceptara sin autenticarla, un atacante en el medio podría reemplazarla por la suya, recibir K y reenviarla cifrada a Bob, leyendo toda la conversación sin que nadie lo note. SecureChat lo impide comparando la huella SHA-256 de la llave recibida con la registrada fuera de banda: en las pruebas, sustituir la llave en tránsito provocó el aborto inmediato del handshake. La protección depende de que la huella se haya verificado correctamente al registrarla. Además, al transportar K con RSA no hay *secreto hacia adelante*: quien obtenga en el futuro la llave privada de Bob podría descifrar sesiones grabadas.

5. Protección de los mensajes

Después del handshake, todo mensaje de usuario se cifra y autentica con AES-256-GCM antes de llegar al socket. No existe ninguna ruta en el código que escriba texto del usuario en claro. La despedida (BYE) también se cifra, para que un atacante no pueda inyectar un cierre falso.

5.1. AES-256-GCM

Tabla 5.1: Parámetros del cifrado simétrico autenticado

Parámetro	**Valor**
Algoritmo	AES-256-GCM (cifrado autenticado, AEAD), clase `AESGCM` de `cryptography`
Clave	La clave de sesión K de 256 bits del handshake
Nonce	96 bits = dirección (1 byte) ‖ contador (11 bytes, big-endian)
Tag	128 bits, concatenado al final del ciphertext
Datos asociados (AAD)	`"SecureChat-v1"` ‖ tipo (1 B) ‖ id de sesión (8 B) ‖ nonce (12 B)
Codificación del texto	UTF-8 (tildes y eñes)
Sobrecosto por mensaje	33 bytes (1 tipo + 12 nonce + 16 tag + 4 de longitud)

5.2. Generación del nonce

Alice y Bob cifran con la *misma* clave K, por lo que cada dirección tiene su propio espacio de nonces. El primer byte indica la dirección (`01`: Alice → Bob, `02`: Bob → Alice) y los once restantes son un contador que crece en uno con cada mensaje.

- Dos nonces de direcciones distintas difieren siempre en el primer byte.
- Dos nonces de la misma dirección difieren siempre en el contador.
- El contador 0 de cada dirección se usa una sola vez, en la confirmación del handshake; el chat empieza en el contador 1.
- Si el contador llegara a $2^{88} - 1$, la sesión termina en lugar de reutilizar un nonce.

Por construcción, ningún nonce se repite con la misma clave. Se descartó el nonce aleatorio de 96 bits: con el volumen de un chat su riesgo de colisión sería despreciable, pero el contador ofrece la garantía por diseño y, además, sirve como número de secuencia.

Por qué el nonce no debe repetirse. En GCM, cifrar dos mensajes con la misma clave y el mismo nonce produce el mismo flujo de cifrado: el XOR de ambos ciphertexts revela el XOR de los plaintexts. Además, permite al atacante recuperar la subclave de autenticación y falsificar tags. El nonce, en cambio, puede viajar en claro: no es secreto, solo debe ser único.

5.3. Verificación en el receptor

El receptor procesa cada registro en este orden, y **solo** muestra el plaintext si todas las comprobaciones pasan:

1. **Formato:** el cuerpo mide al menos 28 bytes (nonce + tag).
2. **Dirección:** el primer byte del nonce debe ser el del par. Un mensaje propio devuelto por un atacante (reflejado) se rechaza.
3. **Secuencia:** el contador debe ser estrictamente mayor que el último aceptado de esa dirección.
4. **Autenticación:** `AESGCM.decrypt(nonce, ct, aad)`; si lanza `InvalidTag`, el mensaje se rechaza y no se muestra nada.

El contador del receptor solo avanza después de un tag válido, de modo que un mensaje alterado no impide aceptar los mensajes legítimos siguientes.

5.4. Protección contra repetición

Como el número de secuencia viaja dentro del nonce y el nonce está autenticado, un atacante no puede cambiarlo sin invalidar el tag. Un registro capturado y reenviado llega con un número ya aceptado y se rechaza. La AAD incluye el tipo de mensaje y el id de sesión, por lo que un registro de chat no puede hacerse pasar por una despedida ni reutilizarse en otra sesión.

5.5. Evidencia en pantalla

El emisor ve su texto plano local y, debajo, lo que realmente sale por el socket: número de secuencia, tamaño de la trama, y nonce, ciphertext y tag en hexadecimal. El receptor ve lo recibido, luego `[TAG OK]` o `[RECHAZADO]` y, solo en el primer caso, el texto recuperado. El hexadecimal es únicamente una forma de *mostrar* bytes; no protege nada.

5.6. Pruebas automatizadas

El archivo `tests/test_secure_channel.py` (13 pruebas con `pytest`) verifica, entre otras cosas, que el plaintext no aparece en los bytes enviados, que 10 000 cifrados consecutivos no repiten nonce, que alterar cualquier byte provoca el rechazo, y que los registros repetidos, reflejados, de otra sesión o con el tipo cambiado se rechazan.

6. Formato de los mensajes transmitidos

TCP entrega un flujo de bytes sin límites de mensaje, por lo que cada mensaje se delimita con un prefijo de longitud de 4 bytes. Las tramas de más de 64 KiB se rechazan antes de leer su contenido.

Tabla 6.1: Estructura de una trama de chat cifrada

Campo	Tamaño	Descripción
Longitud	4 bytes	Tamaño del resto de la trama (big-endian). Visible.
Tipo	1 byte	2 = CHAT, 3 = BYE. Visible, pero autenticado en la AAD.
Nonce	12 bytes	Dirección (1 B) ‖ contador (11 B). Visible, pero autenticado.
Ciphertext	n bytes	Mensaje cifrado; mide lo mismo que el plaintext UTF-8.
Tag	16 bytes	Tag de autenticación de GCM.

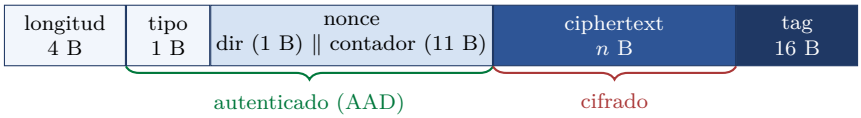


Figura 6.1: Trama de un mensaje de chat. Nada del texto del usuario viaja en claro.

Codificación no es cifrado. La representación en hexadecimal o Base64 se usa solo para mostrar o transportar bytes. No ofrece confidencialidad: cualquiera puede revertirla sin ninguna clave.

7. Conversación bidireccional y cierre de sesión

7.1. Flujo de un mensaje

Mensaje legible → AES-GCM cifra y autentica → ciphertext en tránsito → verificación y descifrado → mensaje recuperado.

[**PENDIENTE:** *Diagrama de secuencia de un mensaje en cada sentido.*]

7.2. Presentación en consola

Tabla 7.1: Distinción visual en la consola

Etiqueta	Significado
[PEND.]	Mensaje local (plaintext antes de cifrar)
[PEND.]	Datos cifrados enviados
[PEND.]	Datos cifrados recibidos
[PEND.]	Mensaje recuperado tras verificar el tag
[PEND.]	Error o mensaje rechazado

7.3. Cierre controlado

[**PENDIENTE:** *Comando de salida, mensaje de cierre cifrado y comportamiento del otro extremo.*]

8. Detección de manipulación

[**PENDIENTE:** *Mecanismo de la prueba controlada (alteración en el emisor o proxy intermedio), qué byte se altera y por qué.*]

[**PENDIENTE:** *Resultado: el receptor detecta el tag inválido, rechaza el mensaje y no muestra plaintext. Política tras el rechazo (descartar o cerrar la sesión).*]

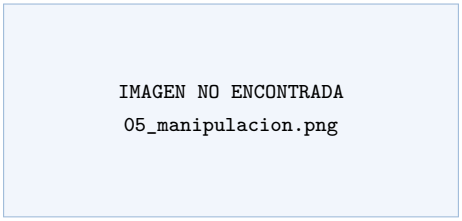


Figura 8.1: Rechazo de un ciphertext alterado.

9. Análisis de seguridad

9.1. Modelo de adversario

[**PENDIENTE:** *Adversario pasivo (escucha el socket) y activo (modifica, repite o inyecta), en la línea del modelo Dolev-Yao visto en clase.*]

9.2. Cumplimiento de los requisitos mínimos

Tabla 9.1: Requisitos mínimos del taller y su cumplimiento

	Requisito	Cómo se cumple	Evidencia
✓	Python 3 y sockets TCP/IP	[PEND.]	[PEND.]
✓	Biblioteca criptográfica reconocida	[PEND.]	[PEND.]
✓	Handshake basado en criptografía asimétrica	[PEND.]	[PEND.]
✓	Clave de sesión dinámica, no fija en el código	[PEND.]	[PEND.]
✓	Ninguna llave privada viaja por el socket	[PEND.]	[PEND.]
✓	Mensajes cifrados con esquema autenticado	[PEND.]	[PEND.]
✓	Nonce correcto y no repetido	[PEND.]	[PEND.]
✓	Rechazo de tags inválidos	[PEND.]	[PEND.]
✓	Evidencia de plaintext, ciphertext enviado y recibido, y plaintext recuperado	[PEND.]	[PEND.]
×	Base64 o hexadecimal no sustituyen al cifrado	[PEND.]	[PEND.]

Continúa en la página siguiente

Tabla 9.1 – viene de la página anterior

	Requisito	Cómo se cumple	Evidencia
×	No se usa ECB	[PEND.]	[PEND.]

9.3. Limitaciones conocidas

[**PENDIENTE:** *Por ejemplo: autenticación de la llave pública, ausencia de secreto hacia adelante (forward secrecy) al transportar la clave con RSA, análisis de tráfico (tamaños y tiempos visibles), y la imposibilidad de borrar con certeza secretos de la memoria en Python.*]

10. Preguntas de cierre

1. **¿Qué problema resuelve la criptografía asimétrica durante el handshake?**
   [**PENDIENTE:** *Respuesta.*]
2. **¿Por qué la conversación utiliza criptografía simétrica después del handshake?**
   [**PENDIENTE:** *Respuesta.*]
3. **¿Qué información puede observar un tercero que capture los paquetes de la conversación?**
   [**PENDIENTE:** *Respuesta.*]
4. **¿Por qué el nonce puede viajar junto al ciphertext y por qué no debe repetirse?**
   [**PENDIENTE:** *Respuesta.*]
5. **¿Qué función cumple el authentication tag?**
   [**PENDIENTE:** *Respuesta.*]
6. **¿Qué ocurriría si la clave de sesión se transmitiera sin protección?**
   [**PENDIENTE:** *Respuesta.*]
7. **¿Qué ocurriría si un atacante obtiene una de las claves privadas?**
   [**PENDIENTE:** *Respuesta.*]

11. Evidencias de la demostración

IMAGEN NO ENCONTRADA
01_conexion_handshake.png

Figura 11.1: Dos instancias en ejecución y handshake completado.

IMAGEN NO ENCONTRADA
02_conversacion.png

Figura 11.2: Conversación con al menos tres mensajes por sentido.

IMAGEN NO ENCONTRADA
03_plaintext_vs_ciphertext.png

Figura 11.3: Comparación entre el mensaje legible y el ciphertext transmitido.

IMAGEN NO ENCONTRADA
04_captura_red.png

Figura 11.4: (Opcional) Captura del tráfico de `localhost`.

[**PENDIENTE:** *Enlace al video de demostración.*]

Referencias

A. Ejecución y estructura del repositorio

[**PENDIENTE:** *Estructura final del repositorio y comandos para instalar, generar llaves e iniciar cada instancia en Windows (resumen del README).*]

B. Declaración de uso de IA

[**PENDIENTE:** *Herramientas usadas, para qué y cómo se verificó el resultado.*]